Развёртывание и оптимизация моделей

Квантизация, серверы инференса и работа в продакшене

Илья Новицкий

12 декабря 2026 · Блок 6

Содержание лекции

- Разберём, из чего складываются задержка, пропускная способность и стоимость генерации.
- Посмотрим на квантизацию и форматы GGUF и AWQ на уровне идей.
- Сравним серверы инференса vLLM и TGI и обсудим интеграцию модели в продукт.
- Обсудим мониторинг и оценку в продакшене, затем сформулируем задачи хакатона.

От ноутбука к сервису

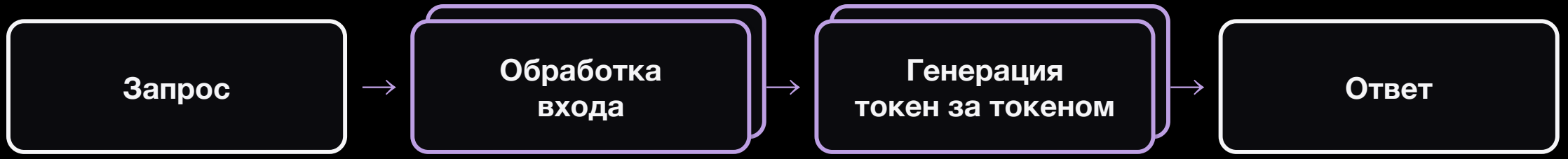
Аспект	Эксперимент в ноутбуке	Сервис
Запросы	Последовательно, по одному	Параллельно, от многих пользователей
Задержка	Не важна	Входит в требования
Сбои	Перезапуск ячейки	Повторы, резервная модель
Качество	Проверено на наборе	Отслеживается постоянно
Стоимость	Разовые вычисления	Растёт с числом запросов

- Модель — одна часть системы; остальное — инфраструктура вокруг неё.

Из чего складывается стоимость

- Внешний API: оплата за входные и выходные токены по тарифу поставщика.
- Собственный сервер: аренда или покупка GPU, электроэнергия и сопровождение.
- Стоимость растёт с длиной контекста: история диалога и документы оплачиваются при каждом запросе.
- Выходные токены обычно дороже входных и генерируются медленнее.

Две фазы генерации



Разные узкие места

- Обработка входа (prefill) считает все входные токены параллельно и заполняет кеш.
- Генерация (decode) добавляет по одному токену; каждый шаг читает веса и кеш из памяти.
- Длинный вход увеличивает время до первого токена, длинный ответ — общее время.

Метрики скорости

Метрика	Что измеряет	Кому важна
TTFT	Время до первого токена	Пользователю чата
TPOT	Время на каждый следующий токен	Пользователю длинных ответов
Задержка	Полное время ответа	Фоновым задачам и API
Пропускная способность	Токенов в секунду на сервер	Владельцу инфраструктуры

- Задержку описывают перцентилями: p50 — типичный запрос, p95 и p99 — медленные.

Оценка полной задержки

$$T \approx TTFT + TPOT \cdot (N_{out} - 1)$$

- N_{out} — число сгенерированных токенов ответа.
- Для чата важнее TTFT: при потоковой выдаче пользователь видит начало ответа сразу.
- Для пакетной обработки важнее пропускная способность, а не задержка одного запроса.
- Обе величины растут при высокой нагрузке на сервер.

Пример расчёта задержки

Сценарий	TTFT	TPOT	Токенов ответа	Итого
Короткий ответ	0,3 с	20 мс	50	≈ 1,3 с
Длинный ответ	0,3 с	20 мс	500	≈ 10,3 с
Длинный документ на входе	2,0 с	25 мс	200	≈ 7,0 с

- Числа условные: они показывают, какая фаза доминирует в каждом сценарии.

Узкое место — память

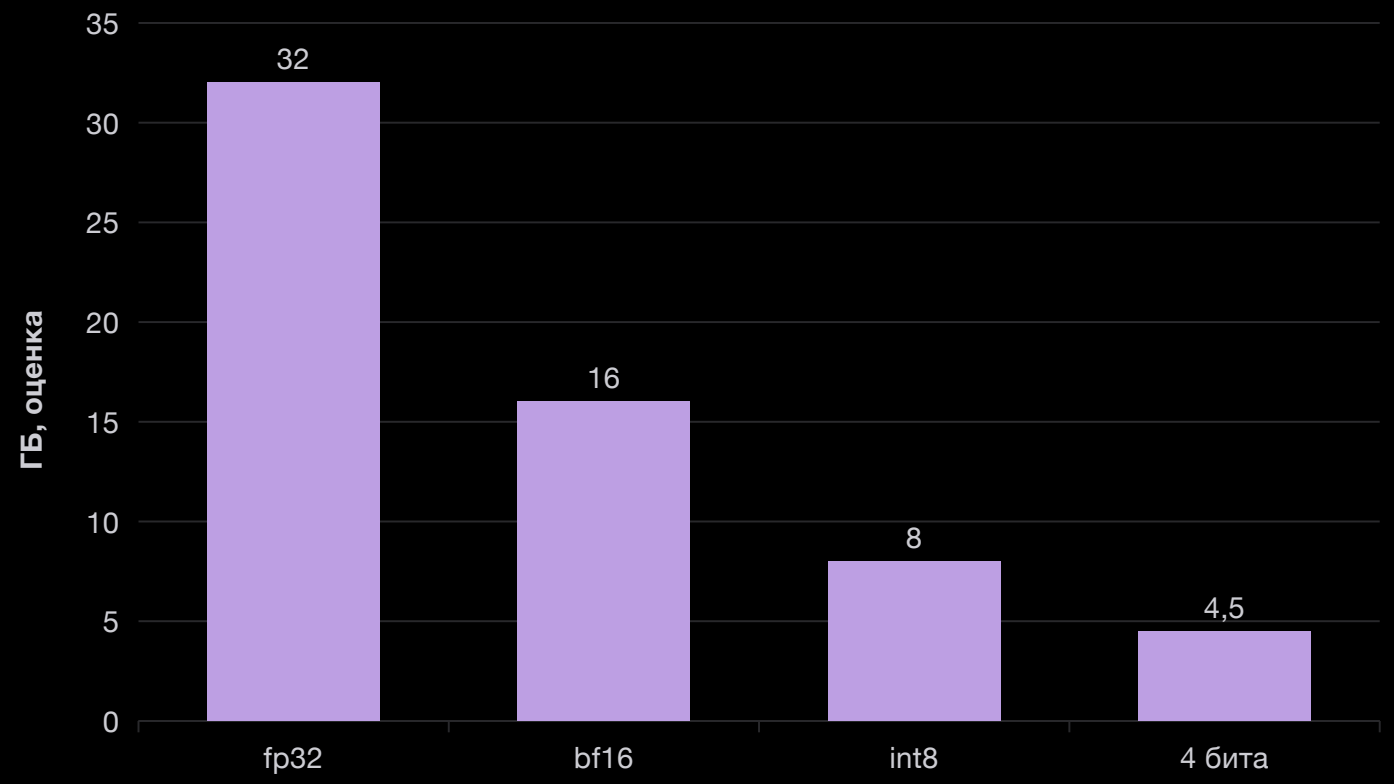
- На шаге генерации нужно прочитать все веса модели ради одного нового токена.
- Скорость шага ограничена пропускной способностью памяти, а не числом операций.
- Поэтому уменьшение размера весов ускоряет генерацию, даже если вычислений не меньше.
- Обработка нескольких запросов одним пакетом использует одно чтение весов для всех.

Память для весов

$$M \approx N \cdot b$$

- N — число параметров, b — байт на параметр в выбранном формате.
- Модель на 8 млрд параметров в bf16 занимает около 16 ГБ только под веса.
- Нужен запас под кеш ключей и значений, активации и служебные буферы.
- Квантизация уменьшает b и позволяет поместить модель на меньшее устройство.

Веса модели на 8 млрд параметров



- Для 4 бит учтены коэффициенты масштаба групп.
- Кеш ключей и значений в оценку не входит.
- Реальный размер файлов зависит от формата и слоёв, оставленных в 16 битах.

Кеш ключей и значений

$$M_{KV} = 2 \cdot L \cdot H_{kv} \cdot d_h \cdot b \cdot T$$

- L — число слоёв, H_{kv} — число голов ключей и значений, d_h — размер головы.
- b — байт на число, T — суммарное число токенов во всех активных запросах.
- Множитель 2 — отдельные ключи и значения.
- Кеш растёт линейно с длиной контекста и числом одновременных запросов.

Пример: кеш для модели класса Llama 3 8B

Параметр	Значение
Слоёв L	32
Голов ключей и значений H_kv	8
Размер головы d_h	128
Байт на число b, bf16	2
На один токен	2 · 32 · 8 · 128 · 2 = 128 КиБ
На 8 192 токена	1 ГиБ на один запрос

- Тридцать одновременных запросов по 8 тысяч токенов требуют больше памяти, чем сами веса.

Квантизация

- Числа весов хранятся с меньшей точностью: вместо 16 бит — 8 или 4.
- Модель занимает меньше памяти и быстрее читается на шаге генерации.
- Округление вносит ошибку; задача методов — сохранить качество на важных весах.
- Чаще всего квантуют уже обученную модель, без повторного обучения.

Как устроено округление

$$q = \mathrm{round}(w / s) + z, \quad \hat{w} = s \cdot (q - z)$$

- s — масштаб, z — смещение нуля; q — целое число в узком диапазоне.
- При 4 битах q принимает только 16 значений.
- Масштаб считают для группы весов, например для 128 соседних чисел.
- Меньшая группа точнее, но требует хранить больше масштабов.

Что квантуют

Вариант	Обозначение	Эффект
Только веса	W4A16, W8A16	Меньше памяти, быстрее генерация
Веса и активации	W8A8, FP8	Ускорение вычислений на подходящих GPU
Кеш ключей и значений	KV в FP8 или int8	Больше запросов и длиннее контекст

- W — веса, A — активации; число — бит на значение.

Когда выполняется квантизация

После обучения

- Нужна обученная модель и небольшой набор примеров для калибровки
- Минуты или часы вычислений
- Основной вариант для готовых моделей

Во время обучения

- Модель учится работать с округлёнными весами
- Требует полноценного обучения
- Лучше сохраняет качество при очень малом числе бит

- Форматы GGUF, AWQ и GPTQ относятся к квантизации после обучения.

Обзор форматов

Формат	Где используется	Особенность
GGUF	llama.cpp, Ollama, LM Studio	Один файл, CPU и GPU, много уровней
AWQ	vLLM, TGI, Transformers	Учитывает активации при выборе масштаба
GPTQ	vLLM, TGI, Transformers	Компенсирует ошибку округления по слоям
bitsandbytes	Transformers, QLoRA	Квантизация при загрузке, без подготовки
FP8	vLLM, TensorRT-LLM	8 бит с плавающей точкой на новых GPU

- Выбор формата определяется сервером инференса и устройством.

GGUF

- Формат файлов проекта llama.cpp: веса, токенизатор и метаданные в одном файле.
- Работает на CPU, Apple Silicon и GPU; подходит для локального запуска и небольших серверов.
- Поддерживает много уровней квантизации — от 2 до 8 бит на вес.
- Готовые файлы часто публикуют рядом с исходной моделью на Hugging Face.

Уровни квантизации GGUF

Обозначение	Бит на вес, около	Типичное применение
Q8_0	8,5	Почти без потерь, большой размер
Q6_K	6,6	Высокое качество
Q5_K_M	5,7	Баланс качества и размера
Q4_K_M	4,8	Частый выбор по умолчанию
Q3_K_M	3,9	Мало памяти, заметные потери

- K — группы с общими масштабами, M — средний вариант по точности важных слоёв.
- Потери качества сильнее у небольших моделей: их проверяют на своей задаче.

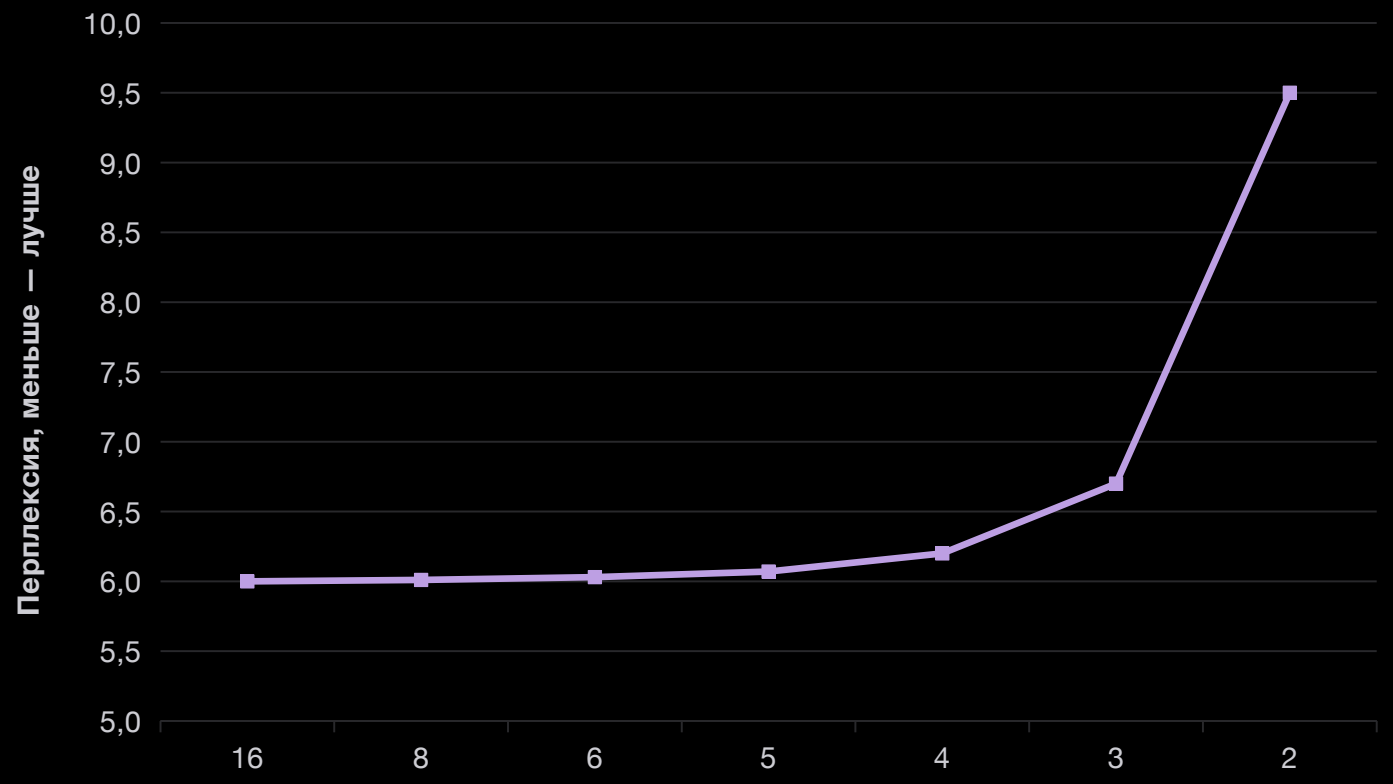
AWQ

- Activation-aware Weight Quantization: квантизация весов с учётом активаций.
- Небольшая доля весовых каналов — порядка процента — сильнее всего влияет на результат.
- Метод находит их по величине активаций на калибровочных примерах и масштабирует перед округлением.
- Результат — 4-битные веса, которые эффективно обслуживаются vLLM и другими серверами на GPU.

GPTQ

- Квантизует веса слой за слоем и корректирует оставшиеся веса, чтобы компенсировать ошибку.
- Использует небольшой калибровочный набор текстов.
- Распространён вместе с AWQ для 4-битных моделей на GPU.
- Качество зависит от калибровочных данных: для русского текста их стоит подбирать.

Качество и число бит



- До 4–5 бит потери обычно невелики.
- Ниже 4 бит качество падает быстро.
- Кривая условная; уровень проверяют на своей модели.

Проверка квантованной модели

- Сравнить с исходной моделью на своём наборе: метрики задачи, а не только перплексию.
- Проверить формат ответов: квантизация иногда ломает строгий JSON и вызовы инструментов.
- Отдельно проверить русский язык и длинный контекст.
- Измерить скорость и память на целевом устройстве с реальной нагрузкой.

Зачем нужен сервер инференса

- Обрабатывает много запросов одновременно и эффективно использует GPU.
- Управляет памятью для кеша ключей и значений и очередью запросов.
- Предоставляет стандартный API, чаще всего совместимый с OpenAI.
- Отдаёт метрики: число запросов в очереди, задержки, использование памяти.

Статическое и непрерывное объединение

Статический пакет

- Запросы ждут, пока соберётся пакет
- Пакет завершается по самому длинному ответу
- Короткие ответы простаивают

Непрерывное объединение

- Новый запрос подключается на любом шаге
- Завершённый запрос сразу освобождает место
- GPU загружен равномернее

- Непрерывное объединение (continuous batching) — стандарт для современных серверов.

PagedAttention

- Кеш ключей и значений хранится блоками фиксированного размера, как страницы памяти.
- Память выделяется по мере роста ответа, а не сразу под максимальную длину.
- Меньше фрагментации — больше одновременных запросов на том же GPU.
- Метод предложен в работе о vLLM, 2023.

vLLM

- Открытый сервер инференса с PagedAttention и непрерывным объединением запросов.
- Поддерживает AWQ, GPTQ, FP8, адаптеры LoRA и кеширование общих префиксов.
- Предоставляет OpenAI-совместимый API и поддерживает много архитектур моделей.
- Основная платформа — GPU NVIDIA; поддержка других устройств развивается.

Запуск vLLM

```
$ vllm serve Qwen/Qwen2.5-7B-Instruct-AWQ \
      --max-model-len 8192 --gpu-memory-utilization 0.9

$ curl http://localhost:8000/v1/chat/completions \
    -H "Content-Type: application/json" \
    -d '{"model": "Qwen/Qwen2.5-7B-Instruct-AWQ",
         "messages": [{"role": "user", "content": "Привет"}]}'
```

- max-model-len ограничивает длину контекста и тем самым память под кеш.
- gpu-memory-utilization задаёт долю памяти GPU, которую сервер займёт.

Text Generation Inference

- Сервер инференса от Hugging Face; распространяется как Docker-образ.
- Поддерживает непрерывное объединение, квантизацию и потоковую выдачу.
- Предоставляет собственный API и OpenAI-совместимый маршрут сообщений.
- Перед выбором сервера проверяют статус поддержки: проекты быстро развиваются.

Запуск TGI

```
$ docker run --gpus all -p 8080:80 \
    -v $PWD/data:/data \
    ghcr.io/huggingface/text-generation-inference:latest \
    --model-id Qwen/Qwen2.5-7B-Instruct --quantize awq

$ curl localhost:8080/v1/chat/completions -d '{...}'
```

- Каталог data сохраняет скачанные веса между перезапусками.
- В продакшене вместо latest указывают конкретную версию образа.

Серверы инференса

Сервер	Сильная сторона	Устройства
vLLM	Пропускная способность, адаптеры	GPU
TGI	Интеграция с Hugging Face	GPU
SGLang	Кеширование префиксов, сложные программы	GPU
llama.cpp, Ollama	Локальный запуск GGUF	CPU, Apple, GPU
TensorRT-LLM	Максимальная скорость на NVIDIA	GPU NVIDIA

- Все перечисленные серверы предоставляют OpenAI-совместимый API.

Один клиент для разных серверов

```
from openai import OpenAI
client = OpenAI(base_url="http://localhost:8000/v1", api_key="local")
response = client.chat.completions.create(
    model="Qwen/Qwen2.5-7B-Instruct-AWQ",
    messages=[{"role": "user", "content": "Привет"}],
    max_tokens=200, stream=True)
```

- Смена сервера или поставщика сводится к адресу и названию модели.
- Это тот же клиент, что на семинаре по работе с API.

Кеширование общего префикса

- Многие запросы начинаются одинаково: системная инструкция, примеры, общий документ.
- Сервер сохраняет кеш для общего начала и не пересчитывает его для следующих запросов.
- Сокращает время до первого токена; поставщики API часто снижают цену таких токенов.
- Чтобы кеш срабатывал, постоянная часть промпта должна идти первой и не меняться.

Спекулятивное декодирование

- Небольшая модель быстро предлагает несколько следующих токенов.
- Большая модель проверяет их за один проход и принимает совпадающую часть.
- Результат совпадает с распределением большой модели, а генерация ускоряется.
- Выигрыш зависит от того, как часто малая модель угадывает продолжение.

Структурированный вывод

- Сервер ограничивает выбор токенов так, чтобы ответ соответствовал JSON-схеме или грамматике.
- Гарантирует корректный синтаксис, но не правильность значений полей.
- В vLLM и других серверах включается параметром запроса.
- Снижает число повторных запросов из-за неверного формата.

Архитектура интеграции

Клиент	Веб-интерфейс, мессенджер, внутренняя система
Бэкенд приложения	Логика, история диалога, поиск документов
Шлюз моделей	Ключи, лимиты, маршрутизация, учёт затрат
Модель	Собственный сервер или внешний API
Журнал и метрики	Запросы, ответы, задержки, оценки

- Шлюз позволяет менять модель и поставщика без изменений в приложении.

Внешний API и собственный сервер

Внешний API

- Быстрый старт, нет инфраструктуры
- Оплата за токены
- Данные передаются поставщику
- Модель может измениться без вашего участия

Собственный сервер

- Контроль над данными и версией
- Оплата за GPU независимо от загрузки
- Нужна команда сопровождения
- Выгоден при стабильно высокой нагрузке

- Часто начинают с API и переходят на свой сервер, когда известны нагрузка и требования.

Стоимость запроса к API

$$C = (N_{in} \cdot p_{in} + N_{out} \cdot p_{out}) / 10^6$$

- p_in и p_out — цены за миллион входных и выходных токенов.
- N_in включает системную инструкцию, историю и документы — при каждом запросе.
- Кешированные входные токены у многих поставщиков стоят дешевле.
- Месячная стоимость = средняя стоимость запроса × число запросов.

Пример расчёта стоимости

Сценарий	Вход	Выход	Цена запроса	100 тыс. запросов
Короткий вопрос	300	150	$0,00026	$26
Ответ по документам	4 000	300	$0,00138	$138
Длинный диалог	12 000	400	$0,00350	$350

- Условные цены: $0,25 за миллион входных и $1,25 за миллион выходных токенов.
- Основной рост — из-за входа: документы и история оплачиваются в каждом запросе.

Стоимость собственного сервера

$$C_{1m} = P_час / (R \cdot 3600) \cdot 10^6$$

- P_час — стоимость часа работы сервера, R — токенов в секунду при реальной нагрузке.
- Пример: $2 в час и 1 000 токенов в секунду дают около $0,56 за миллион токенов.
- При низкой загрузке R падает, и цена за токен растёт: простой GPU тоже оплачивается.
- К расчёту добавляют сопровождение, резервирование и время инженеров.

Как снизить задержку

- Потоковая выдача: пользователь видит начало ответа, пока генерируется остальное.
- Короче вход: убрать лишние документы и старую историю, сжать инструкцию.
- Меньше модель или квантизация — если качество на задаче сохраняется.
- Кеширование префикса и повторяющихся ответов.

Маршрутизация между моделями

- Простые запросы обрабатывает небольшая быстрая модель, сложные — большая.
- Решение принимает классификатор, правило или сама малая модель по уверенности.
- Резервная модель отвечает, если основная недоступна или превысила время ожидания.
- Маршрутизацию оценивают так же, как модель: долей правильно направленных запросов.

Кеширование ответов

- Точный кеш: одинаковый запрос и параметры возвращают сохранённый ответ.
- Семантический кеш: похожий по смыслу запрос получает ответ на ранее заданный вопрос.
- Семантический кеш может вернуть ответ на другой вопрос; порог сходства проверяют.
- Кеш не подходит для персональных данных и быстро меняющейся информации.

Надёжность

- Тайм-ауты на каждый вызов и ограничение общей длительности обработки запроса.
- Повторы с увеличивающейся паузой при временных ошибках, но не при ошибках запроса.
- Лимиты запросов на пользователя, чтобы один клиент не занял весь сервер.
- Понятное сообщение пользователю, если ответ получить не удалось.

Что отслеживать

Уровень	Метрики
Инфраструктура	Загрузка GPU, память, очередь, ошибки
Модель	TTFT, TPOT, длина входа и ответа, причины остановки
Качество	Оценки пользователей, оценки судьи на выборке, доля отказов
Продукт	Решённые обращения, повторные вопросы, стоимость на пользователя

- Технические метрики показывают, что система работает; продуктовые — что она полезна.

Журнал запросов

- Сохраняют запрос, ответ, версию модели и промпта, параметры генерации и время.
- Для поиска по документам — идентификаторы найденных фрагментов.
- Персональные данные маскируют или не сохраняют; срок хранения ограничивают.
- Журнал — основной источник данных для оценки и следующего дообучения.

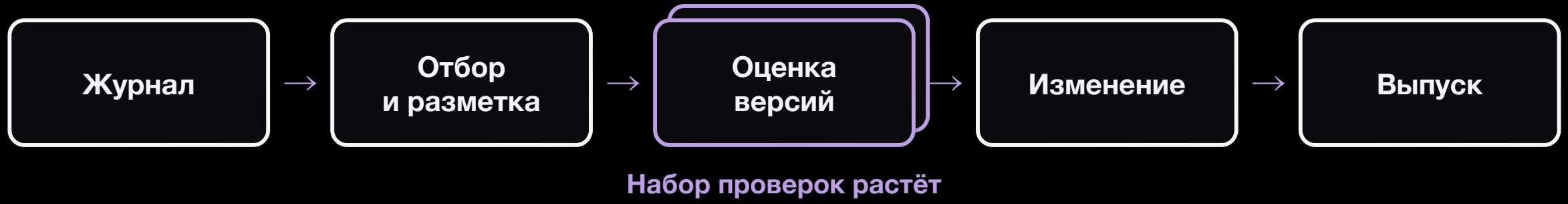
Оценка в продакшене

- Явные сигналы: оценка ответа пользователем, жалобы, обращения к оператору.
- Неявные сигналы: повторный вопрос, копирование ответа, длительность сессии.
- Модель-судья оценивает случайную выборку ответов по фиксированным критериям.
- Регулярно часть ответов читают люди — для калибровки автоматических оценок.

Изменение данных со временем

- Пользователи начинают задавать другие вопросы: новые продукты, сезонные темы.
- Документы обновляются, а индекс поиска — нет.
- Внешний API меняет версию модели, и поведение меняется без изменений в коде.
- Признаки — рост доли отказов, повторных вопросов и снижение оценок судьи.

Цикл улучшения



- Ошибки из журнала превращаются в новые тестовые примеры.
- Изменением может быть промпт, индекс документов, модель или адаптер.
- Новая версия сравнивается с текущей на одинаковом наборе до выпуска.

Безопасный выпуск

- Теневой режим: новая версия обрабатывает копию запросов, ответы не показываются.
- Постепенный выпуск: новая версия получает небольшую долю трафика.
- A/B-сравнение по продуктовым метрикам на сопоставимых группах пользователей.
- Возможность быстро вернуться к предыдущей версии модели и промпта.

Чек-лист перед запуском

Вопрос	Проверено
Качество на своём наборе выше базовой линии?	Отчёт об оценке
Задержка p95 в пределах требований?	Нагрузочный тест
Стоимость на месяц посчитана?	Расчёт по токенам
Защита от атак и утечек?	Набор атак, фильтры
Журнал и метрики настроены?	Панель мониторинга
Есть откат к прежней версии?	Процедура отката

Хакатон: формат

- Команды по 3–4 человека собирают работающий прототип прикладной системы на основе LLM.
- Один день: сразу после лекции, 12:00–15:10 — разработка, оценка и защита.
- Решение должно включать оценку качества на наборе проверок, а не только демонстрацию.
- Можно использовать материалы и код всех семинаров курса.

Треки хакатона

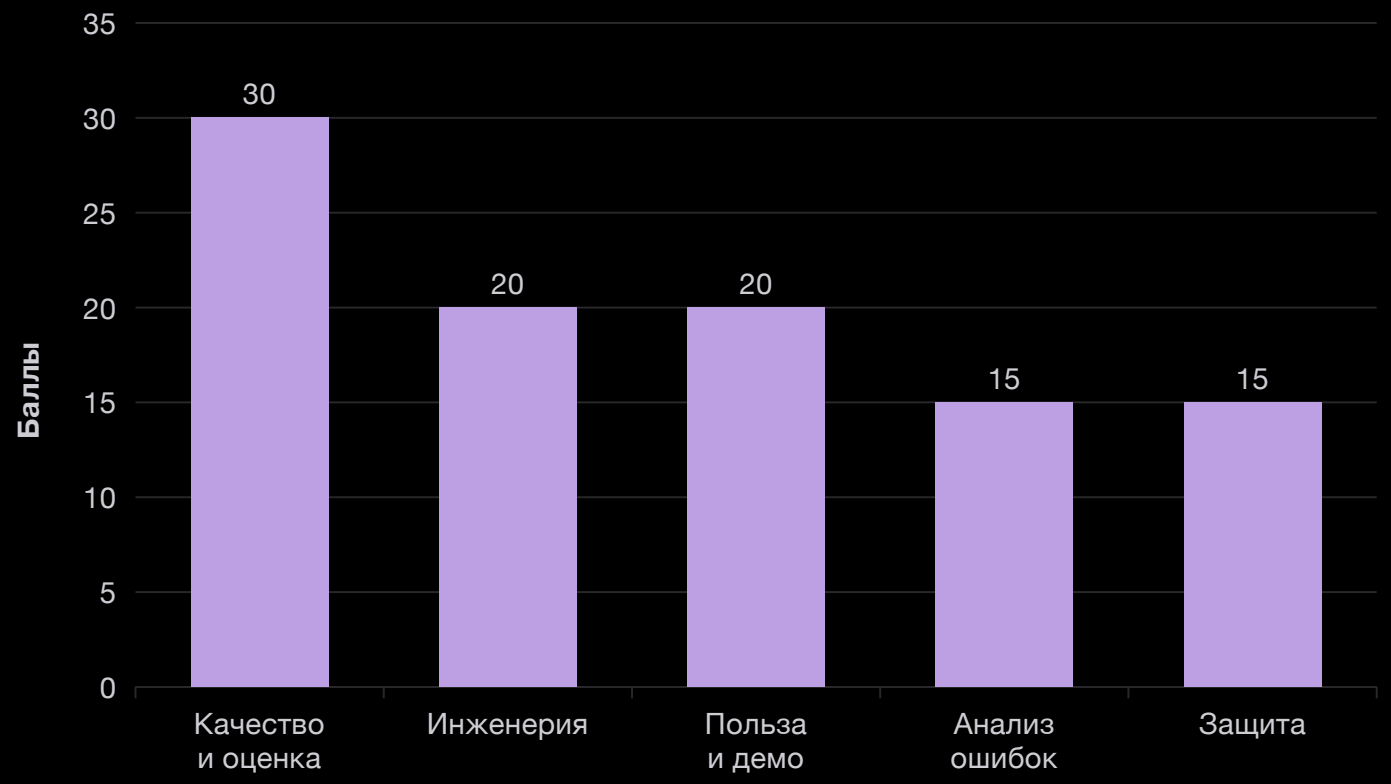
Трек	Что собрать	Обязательная оценка
Ассистент по документам	Поиск, ответ со ссылками	Поиск и соответствие источнику
Чат-бот поддержки	Классификация, извлечение, ответ	Точность и формат
Дообученная модель	LoRA для узкой задачи	До и после, попарно
Агент с инструментами	Поиск, расчёты, база данных	Доля решённых задач
Быстрый сервис	Квантизация и сервер	Качество, задержка, стоимость

- Своя тема возможна по согласованию до начала хакатона.

Требования к решению

- Репозиторий с инструкцией запуска, фиксированными версиями библиотек и описанием данных.
- Набор проверок не меньше 20 примеров, собранный до экспериментов.
- Сравнение с базовой линией: например, промпт без поиска или исходная модель.
- Демонстрация работы и короткий отчёт: результаты, ошибки, ограничения, стоимость.

Критерии оценки



- Сумма — 100 баллов.
- Качество оценивается по честности проверки, а не только по итоговому числу.
- Инженерия — воспроизводимость, структура кода, обработка ошибок.

План хакатона, 12 декабря

Время	Что делаем
12:00–12:20	Команды, выбор трека, данные
12:20–13:00	Набор проверок и базовая линия от входа до ответа
13:00	Контрольная точка: базовая линия и первая оценка
13:00–14:10	Улучшения и сравнение версий
14:10–14:30	Отчёт и подготовка демонстрации
14:30–15:10	Защита: 5 минут доклада и 2 минуты вопросов

- Команды, трек и данные лучше выбрать заранее — до дня хакатона.

Как не потерять время

- Сначала — самая простая работающая версия от входа до ответа, затем улучшения.
- Набор проверок собирают в первые 40 минут: без него нельзя сравнить варианты.
- Модели и данные скачивают заранее; для тяжёлых моделей используют API или квантизацию.
- Каждое изменение фиксируют вместе с результатом оценки.

Контрольные вопросы

- Почему генерация токенов ограничена памятью, а не вычислениями?
- Сколько памяти займёт кеш для 16 запросов по 4 000 токенов у модели из примера?
- Чем AWQ отличается от простого округления весов до 4 бит?
- Почему непрерывное объединение запросов увеличивает пропускную способность?
- Какие метрики покажут, что качество системы в продакшене ухудшилось?

Материалы

- До 12 декабря: собрать команду, выбрать трек и проверить доступ к модели.

Efficient Memory Management for LLM Serving with PagedAttention

AWQ: Activation-aware Weight Quantization

Документация vLLM

Документация Text Generation Inference

llama.cpp и формат GGUF